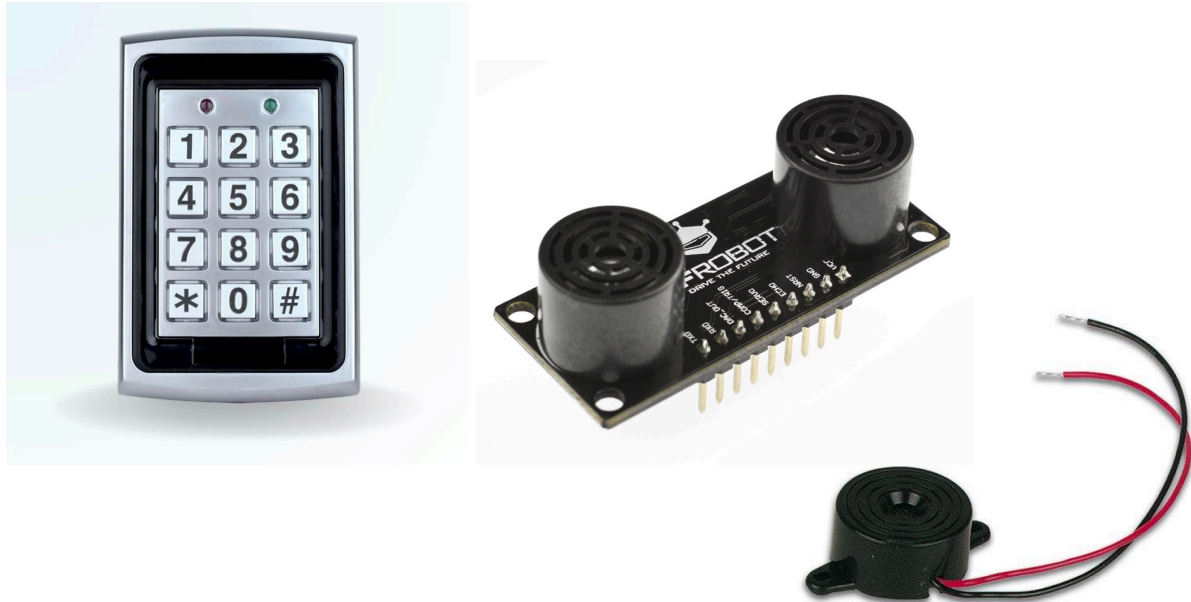




Compte Rendu SAé Alarme Ultra Son

Sommaire :

- I. Cahier des charges (p. 3)*
- II. Synoptique (p. 4)*
- III. Partie matériel (p. 5)*
- IV. Partie logiciel (p. 7)*
- V. Vérifications (p. 13)*
- VI. Bilan (p. 15)*
- VII. Annexes (p. 16)*

I. Cahier des Charges :

Réalisation d'une **alarme**, activable et désactivable avec un **code à 4 chiffres** à entrer sur un **clavier**.

Un usage classique dans la vie courante serait lorsqu'un habitant quitte sa maison, il saisit le code **4582** sur le clavier ce qui enclenche l'alarme. Une **LED s'allume** pour signifier que l'alarme est bien enclenchée. Pour enlever l'alarme en revenant chez lui, l'habitant doit saisir de nouveau le code 4582. La LED s'éteint afin de signifier que l'alarme n'est plus enclenchée.

Lorsque l'alarme est enclenchée : Si le détecteur **URM37** détecte la présence d'une personne, une **sirène** devra retentir. La commande de la sirène sera réalisée avec l'**Atmega 328** qui émet un signal qui varie de 500 à 1 kHz puis retour à 500 Hz, imitant un sirène retranscrite de manière sonore à l'aide d'un buzzer.

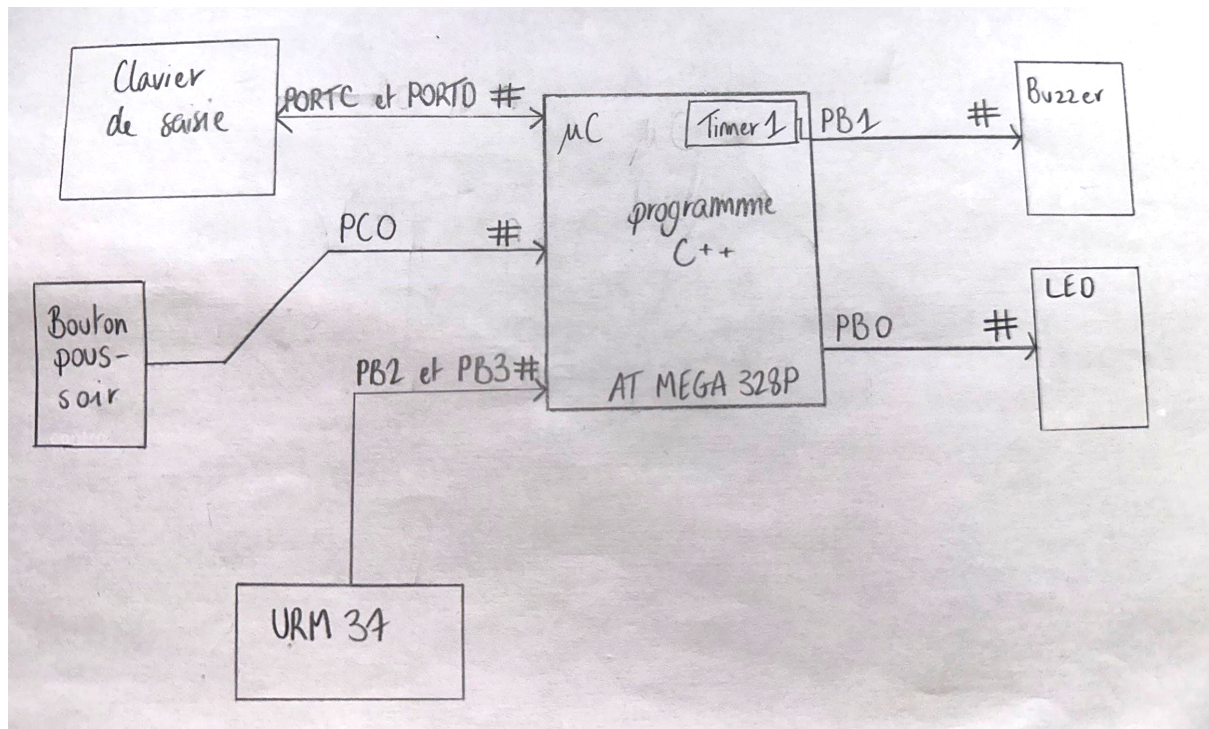
Enfin, le code doit être **modifiable** en appuyant sur un **bouton poussoir** et en tapant en même temps le nouveau code (alarme non enclenchée). La **led** doit **clignoter** 2 fois afin de signifier que le nouveau code a bien été pris en compte.

Cette réalisation contient :

- *Modèle Ultra Son URM37 (imposé)*
- *Clavier 12 touches contenant résistances de tirages (imposé)*
- *Buzzer*
- *Led*
- *arduino ATMEGA328P (imposé)*
- *carte fille GEL pour simplifier l'utilisation des ports du μC (imposé)*
- *bouton poussoir (sur la carte fille GEL)*

Nous avons rajouté à notre réalisation une **LED supplémentaire** qui nous informe que l'alarme doit être déclenchée (détection du capteur US) pour avoir un retour direct si notre buzzer ou si le timer est défectueux.

II. Synoptique :



: signal numérique (0/1L)

III. Partie Matériel :

Traitement des informations :

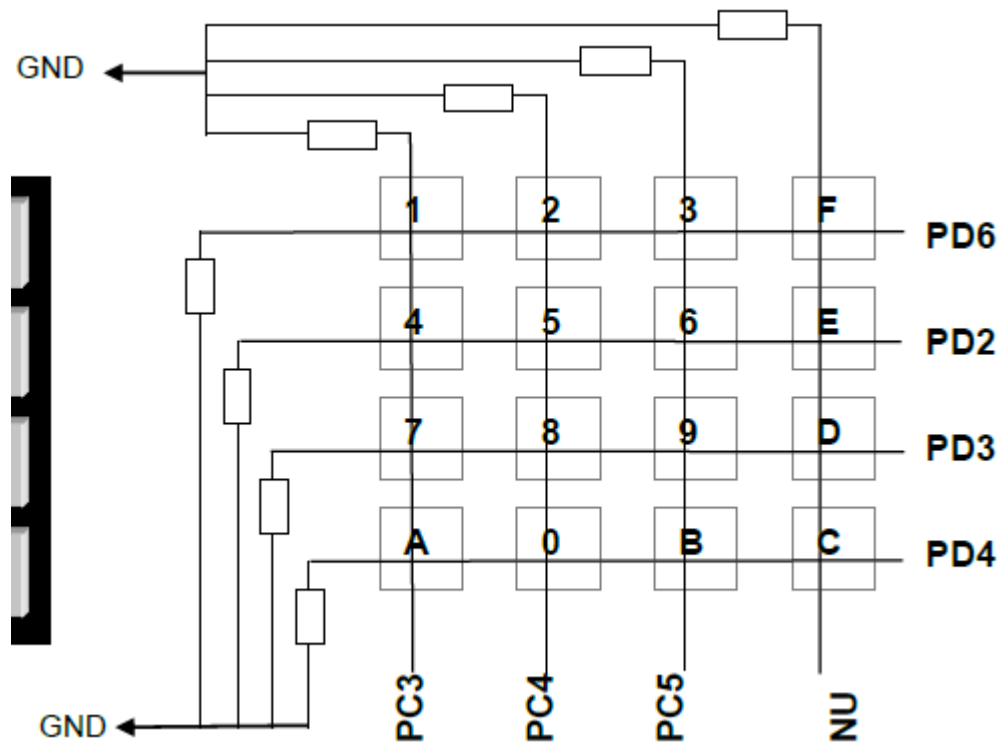
⇒ *Microcontrôleur ATmega328 + Carte Fille GEII*

Le **bouton poussoir** utilisé pour changer le code secret de l'alarme est directement intégré à la carte fille GEII, connecté au port **PC0**.

Le câblage du bouton est réalisé en **actif bas**, donc un niveau logique **0L** sur PC0 correspond à un **appui de le BP**.

Clavier Matriciel :

Le **clavier matriciel** est directement connecté à la **carte fille GEII** et relié aux ports de l'Arduino de la manière suivante :



Le câblage est réalisé en **actif haut**, donc un niveau logique **1L** sur un port correspond à un **appui de la touche**.

La colonne de droite n'est pas utilisée dans notre projet

Utilisation d'une plaque Labdec pour connecter les leds, le buzzer et le capteur US

Câblage des LEDs :

Pas besoin de résistance pour limiter le courant dans la **LED** en sortie du pin de l'arduino car le courant en sortie est déjà très faible (*de l'ordre de 20 mA*) et présence d'une **résistance interne** au microcontrôleur pour éviter de détériorer les composants.

La LED utilisée pour **prévenir de l'activation de l'alarme** et du changement d'un nouveau code est câblé sur la sortie **PB0**

La LED utilisée pour avoir un retour de la **détection de la présence** par l'alarme ultrason est câblé en **parallèle du buzzer** sur la sortie **PB1**

Pour les LEDs, nous connectons l'**Anode** sur la sortie de la carte Arduino et la **cathode** à la masse commune à celle de l'Arduino.

Câblage du BUZZER :

Le **buzzer** est câblé sur la sortie **PB1** (*sortie Output Compare du Timer 1*) pour lui permettre de sonner. Il est connecté sur son autre patte à la masse de la carte Arduino.

Câblage du capteur US :

En nous référant à la **documentation technique du capteur** (cf Annexes), nous l'avons câblé de la manière suivante :

- *GND : Masse de la carte Arduino*
- *Vcc : Sortie 5V de la carte Arduino*
- *ECHO : Sortie PB2*
- *COMP/TRIG : Sortie PB3*

IV. Partie logiciel :

Notre **programme** a été conçu sous forme d'un **graphe d'état**. Cela nous a permis de gérer les différentes phases de fonctionnement de l'alarme et ainsi d'obtenir un programme **clair** dans lequel nous avons incorporé de nombreuses fonctions.

⇒ Retrouvez le graphe d'état global de notre système en Annexes :

Configuration des registres :

La configuration s'effectue dans la fonction setup de notre fichier .ino

- *Initialisation de la liaison série à une vitesse de 9600 bits par seconde*

Serial.begin(9600);

- *Initialisation du timer*

En nous référant à la **documentation technique du Timer 1** de l'ATMega328 (cf Annexes)
Nous configurons le Timer 1 de la manière suivante :

Mode PWM Phase and Frequency Correct avec val max comptage ICR1

Les bits ICNC1 et ICES1 sont positionnés à 0

prescaler = 1

ICR1 ⇒ Valeur max de comptage

Nous obtenons :

TCCR1A = 0x80; //0b10000000

TCCR1B = 0x00; //0b00000000 , nous démarrons notre système avec la timer désactivé

TCCR1B = 0x11; //0b00010001 //allumer l'alarme // activer le timer

ICR1 = 8000000 / (2 * 1 * 500); // TOP (Fclk/(2*prescaler*fréquence))

OCR1A = 0.5 * ICR1; // 50 % (rapport cyclique)

- *Initialisation des ports entrée/sortie*

DDRB = 0x0B; //0b 0000 1011

Bit 0 du PortB en sortie sur laquelle est câblée la **LED** qui confirme l'activation de l'alarme et le nouveau code.

Bit 1 du PortB en sortie sur laquelle est câblée le **Buzzer** commandés par le Timer 1 avec OC1A (Output compare Timer 1)

Bit 2 du PortB en entrée sur laquelle est câblée le port **ECHO** du capteur ultrason URM37.

Bit 3 du PortB en sortie sur laquelle est câblée le port **COMP/TRIG** du capteur ultrason URM37.

Les autres Bits du PortB sont donc configurés en entrée.

- *Initialisation du graphe d'état*

`*p_etat = ALARME_OFF;`

Le programme démarre dans l'état **ALARME_OFF**, qui correspond à alarme désactivé et attente que les touches soient relâchées.

Détail du fonctionnement du graphe d'état

Le code fonctionne avec 5 fichiers :

- **fonctions_ultrason.h** : contient type énumérée TEtat_US liée à l'information que renvoie le URM37 et les prototypes des fonctions utilisées dans le programme.
- **graphe_ultrason.h** : contient type énumérée TEtat liée au graphe d'état principale et les prototypes des fonctions du graphe d'état.
- **fonctions_ultrason.cpp** : contient les fonctions utilisées dans le programme.
- **graphe_ultrason.cpp** : contient les fonctions du graphe d'état et les variables globale du code.
- **main_ultrason.ino** : setup() et loop() et appel des fichiers qui déclarent les fonctions d'abstraction du matériel (#include "graphe_ultrason.h" et #include "fonctions_ultrason.h")

L'état courant et les **entrées** de notre graphe d'état sont passés en **paramètres** aux fonctions. Ils sont souvent passés par adresse, sous forme de **pointeurs**. Nous évitons de recopier tout le contenu de la structure à chaque appel, ce qui gagne de la place et du temps.

Cela nous a permis d'avoir un **programme** final ultra **lisible** et un main qui comporte uniquement 5 lignes.

Fonctionnement général du graphe d'états

Le fonctionnement du système débute dans l'état **ALARME_OFF**, qui constitue l'état initial de notre système. Dans cet état, l'alarme est désactivée, les LEDs sont éteintes et le timer est arrêté. Le système réinitialise également la distance de référence mesurée par le capteur ultrason.

Lorsque aucune touche est pressée puis relâchée, le relâchement provoque un passage dans l'état **ATTENTE_ON**. C'est depuis ce mode que l'utilisateur peut changer le code s'il appuie sur le BP de la carte fille. Sinon le déroulement classique nous envoie en mode **MEMO_ON**.

(Les deux passages sont évidemment exclusifs) Cet état sert à enregistrer les touches saisies par l'utilisateur dans un tableau mémoire. Chaque touche est comparée au code correct. La saisie fonctionne par écrasement sur l'ancienne valeurs pour constamment garder un code à 4 chiffres.

A chaque nouvelle saisie le système passe dans l'état **VERIF_CODE_ON**. Cet état effectue la vérification finale du code. Si le code est valide, le système poursuit vers l'état **INIT_TAB_ON**. Dans le cas contraire, il retourne directement dans l'état **ALARME_OFF**, continuant la boucle.

L'état **INIT_TAB_ON** réinitialise le tableau utilisé pour la saisie du code. Le capteur ultrason détermine la distance référence (la distance face à lui) puis passe automatiquement dans l'état **ALARME_ON**.

L'état **ALARME_ON** correspond au mode de surveillance. La LED de signalisation est allumée et la distance mesurée par le capteur ultrason est continuellement comparée à la distance de référence mémorisée précédemment. Si une variation importante est détectée, cela signifie qu'un objet ou une personne est entré dans la zone surveillée. Le buzzer est alors activé afin de signaler l'intrusion. Tant que l'utilisateur n'agit pas, l'alarme reste dans cet état.

Pour désactiver l'alarme, l'utilisateur doit saisir le bon code. Le système passe alors dans l'état **ATTENTE_OFF**, qui attend la première touche du clavier. Lorsqu'une touche est détectée, le système entre dans l'état **MEMO_OFF**.

L'état **MEMO_OFF** fonctionne de manière similaire à **MEMO_ON**. Les touches saisies sont enregistrées dans un tableau puis comparées au code pour vérifier.

Le système passe ensuite dans l'état **VERIF_CODE_OFF**. Si le code saisi est correct, la transition se fait vers l'état **INIT_TAB_OFF**. Si le code est erroné, le système retourne immédiatement dans l'état **ALARME_ON** et la surveillance continue, le buzzer peut sonner ou non en fonction de la détection ou pas.

L'état **INIT_TAB_OFF** réinitialise les variables de saisie du clavier puis retourne automatiquement vers l'état **ALARME_OFF**. Le système est alors totalement désarmé et revient dans sa configuration initiale.

Modification du code secret

Depuis l'état **ATTENTE_ON**, l'utilisateur peut appuyer BP afin d'accéder au mode de modification du code. Le système entre alors dans l'état **NOUVEAU_CODE**.

Dans cet état, chaque touche saisie est enregistrée dans un tableau contenant le futur code. Après chaque pression, le système passe dans l'état **NOUVEAU_CODE_ATTENTE** afin d'attendre que la touche soit relâchée pour la "retenir" dans l'état **NOUVEAU_CODE_MEMO**, cette nouvelle valeur remplace l'ancienne à l'indice correspondant dans un tableau **tab_solution[]**, s'ensuit une incrémentation d'un compteur puis le retour en mode **ATTENTE_ON**.

Une fois le chiffre sauvegardé, le compteur vaut 4, le système passe dans l'état **NOUVEAU_CODE_SAUV**. L'index de saisie est remis à zéro et une LED clignote pour confirmer l'enregistrement du nouveau code.

Enfin, le système retourne dans l'état **ATTENTE_ON**, prêt à fonctionner avec le nouveau code de sécurité.

Outils supplémentaire intégrées au graphe d'état global :

- *Vérification du code tapé par l'utilisateur avec la solution*

A chaque appui sur le clavier matriciel, le programme enregistre la valeur de la touche dans le tableau **tab_touches[]**.

Le programme regarde ensuite si la combinaison des 4 dernières touches appuyées par l'utilisateur et stockée dans le tableau **tab_touches[]** correspond à la **solution**

//Vérification si le code entré correspond à la solution, code_correct = 1 si le code correspond

```
code_correct = true;
for (int i = 0; i < 4; i++) {
    if (tab_touches[i] != solution[i]) {
        code_correct = false;
    }
}
```

Lorsque le booléen **code_correct = 1**, l'alarme s'active ou se désactive, en fonction de l'endroit où l'utilisateur se situe dans le graphe d'état.

- *Création d'un nouveau code*

Lorsque l'utilisateur se trouve dans l'état **ATTENTE_ON** avec l'alarme désactivée (pas de touche appuyée sur le clavier matriciel), il a la possibilité de créer un nouveau code en appuyant sur le bouton Poussoir de la carte fille GELI Arduino. Dans ce cas, le programme va alors attendre que l'utilisateur appuie sur 4 touches successivement. Une fois que

l'utilisateur a appuyé sur les 4 touches, la solution de l'alarme est remplacée par ce nouveau code et le programme retourne dans l'état **ATTENTE_ON**.

Pendant que le programme est dans l'état de sauvegarde de la nouvelle solution, on va faire clignoter la **LED deux fois** pour prévenir l'utilisateur que son nouveau code a bien été pris en compte. Pour faire clignoter la **LED**, nous avons utilisé la fonction Arduino **delay()**. C'est une fonction bloquante, mais ce n'est pas dérangeant dans cette partie du programme car l'alarme est désactivée donc nous n'avons pas besoin de vérifier en permanence la distance du capteur ultrason.

Présentation des fonctions :

- *Récupération des touches pressées*

Lorsque l'on veut détecter sur quelle touche l'utilisateur est en train d'appuyer, on va d'abord chercher à savoir sur quelle ligne la touche se trouve, puis, sur quelle colonne la touche se trouve.

Tout d'abord, nous configurons les **colonnes** du clavier en **sortie** (DDRC : 1L) et les **lignes** en **entrées** (DDRD : 0L).

Puis pour lire la lignes ou se trouve la touche, on impose un 1L sur les sorties (colonnes = PORTC)

La configuration est ensuite inversée afin d'identifier précisément la colonne ou se trouve la touche.

Utilisation de la fonction **delay()** sur un court instant pour s'assurer que la configuration des ports soit faite avant de lire la valeur des ports en sortie.

delay(1); //delay très court nécessaire pour s'assurer que la configuration soit faite

Une fois la valeur de la ligne et de la colonne correspondant à la touche obtenus, nous nous référons à un tableau pour retrouver la valeur de la touche

//on retrouve la valeur du clavier correspondant en associant la ligne et la colonne

val = tab[line_value][column_value];

- *Récupération de la distance de l'ultrason*

La fonction de récupération de la valeur de la distance du capteur ultrason a été codée sous la forme d'un **graphe d'état** : cf Annexes

Le capteur fournit un signal **ECHO** dont la durée à l'état bas est **proportionnelle à la distance**.

Relation :

Distance (cm) = Δt / 50

avec : Δt en μs .

Nous avons donc cherché à créer une fonction pour mesurer la durée de l'état bas du signal reçu sur la sortie ECHO.

Lorsque la fonction s'exécute, on "réveille" le module ultrason en mettant à 1L l'entrée COMP/TRIG (câblée sur le port PB3). Puis à la fin de la fonction, on place le capteur en veille pour éviter qu'il ne consomme.

On effectue la mesure dans un **do{ }while**; pour s'assurer de mesurer la durée totale de l'état bas sur la sortie ECHO car on est alors obligé de passer dans toutes les étapes de notre graphe d'état.

Nous venons récupérer l'état de la sortie ECHO grâce à la valeur lue sur PB2.

Ensuite, grâce à la fonction Arduino **micros()** (non bloquante), nous pouvons faire la différence entre le temps à l'état haut et celui à l'état bas pour récupérer la durée totale de l'état bas.

La fonction convertit cette durée en distance qui est retournée.

```
uint16_t gestion_ultrason(TEtat_US *g_etat_us) {
PORTB |= BIT3;      //Réveiller le module ultrason (l'entrée COMP/TRIG)
do {
    [...]
    us_distance = (t2 - t1) / 50;
    //On fait la différence pour obtenir la durée de l'état bas, puis on divise par 50
    pour obtenir la distance correspondante en cm.
}
break;
}
} while (*g_etat_us != US_HIGH);
//On effectue la mesure dans un do{ }while pour s'assurer de mesurer la durée totale de l'état
bas sur la sortie Echo
PORTB &= ~BIT3;      //Placer le module ultrason en veille (l'entrée COMP/TRIG)
return us_distance;
}
```

- *Faire sonner l'alarme (sirène)*

Notre fonction **alarme()**; est activée à chaque passage de **loop()**;

Le buzzer va sonner selon la valeur du registre **TCCR1B** (Timer activé ou non).

Pour générer cette sirène, nous avons fait **osciller** la **fréquence** de notre timer entre 500 Hz et 1000Hz. La vitesse de l'oscillation de la sirène est dépendante de la vitesse d'exécution de notre programme. A chaque passage du **loop()**; la fréquence du timer est **incrémentée ou décrémentée de 3 Hz**, en fonction du sens dans lequel il faut aller. Le sens est défini par les seuils de fréquences (500 - 1000Hz). Lorsque la fréquence dépasse 1000 Hz, le sens s'inverse jusqu'à 500 Hz et inversement.

La fonction modifie bien à chaque fois la valeur de **ICR1** avec la nouvelle fréquence obtenue, ainsi que la valeur de **OCR1A** pour conserver un rapport cyclique de **50%**.

V. Explications sur la mise en place des tests et des vérifications de bon fonctionnement :

Nous avons testé **chaque fonction séparément** avant de les incorporer dans notre code principale.

Pour **tester** et **vérifier** le bon fonctionnement de nos fonctions avant implémentation dans notre système, nous avons principalement utilisé le **moniteur série** (`Serial.println();`) grâce à la liaison série avec la carte Arduino.

Nous n'avons pas oublié de retirer les print car ce sont des **fonctions bloquantes** qui perturberaient notre code

Test du fonctionnement du clavier

Pour tester le bon fonctionnement du clavier matriciel, nous avons codé un programme conçu pour récupérer la valeur des touches qui sont pressées. Nous avons renvoyé le résultat obtenu sur le moniteur série pour avoir un retour direct de la valeur de la touche. Une fois que la valeur obtenue était correcte, nous avons pu incorporer ce programme dans une fonction, et la rajouter dans notre programme principale. La fonction est appelée lors de l'appel de la lecture des entrées. Donc cette fonction est exécutée à chaque tour de `loop()` ; .

Test du fonctionnement du capteur ultrason

Après avoir codé notre programme conçu pour récupérer la distance mesurée par le capteur ultrason, nous avons utilisé le moniteur série pour afficher cette distance. Nous avons aussi connecté la sortie ECHO à un oscilloscope pour pouvoir mesurer la durée de l'état bas et la comparer avec la valeur affichée par le moniteur série. Avec notre main, nous faisons varier la distance de détection du capteur et pouvions contrôler en direct les changements sur le moniteur série et sur l'oscilloscope. Une fois le programme fonctionnel, et une valeur de distance obtenue cohérente, nous avons cherché à ajouter un seuil de variation de distance. Ainsi, lorsque l'on détecte une variation de plus de 5cm dans la distance mesurée par le capteur US, l'alarme se déclenche. Nous avons utilisé une LED pour simuler le fonctionnement de l'alarme.

Nous avons ensuite pu incorporer ce programme dans une fonction, et la rajouter dans notre programme principale. La fonction est appelée lors de l'appel de la lecture des entrées. Donc cette fonction est exécutée à chaque tour de `loop()` ; .

Test du fonctionnement du Timer + Buzzer Sirène

Tout d'abord, il a fallu configurer correctement les Registre du Timer 1 de sorte à générer un signal PWM et ainsi commander le buzzer. Une fois que le buzzer faisait du bruit, nous avons cherché à faire varier sa fréquence. Dans un premier temps en le faisant alterner entre 2 fréquences. Une fois que cela fonctionnait, sans utilisation de fonction `delay()` car

bloquant, nous avons cherché à générer un signal avec une fréquence qui varie progressivement comme une sirène.

Pour contrôler la fréquence de notre signal, nous avons pu utiliser un oscilloscope que nous connectons à la sortie de l'arduino pour visualiser le signal en sortie du Timer 1.

Une fois le fonctionnement correct de notre buzzer obtenu, nous avons pu incorporer ce programme dans une fonction, et la rajouter dans notre programme principale. La fonction est appelée lors du loop(). Donc cette fonction est exécutée à chaque tour de loop(); et le buzzer sonne lorsque le timer est activé.

Validation du programme au globale

Scénario complet :

1. Saisie du code 4582.
2. Activation de l'alarme (LED ON).
3. Détection d'une personne.
4. Déclenchement du buzzer.
5. Désactivation par saisie du code.
6. Création nouveau code.

VI. BILAN :

A pretty good aspect of the project is that the system is precise and has a **real-life utility**. It could easily be used by an **individual** at home or in a **professional** installation.

Throughout this project, we learned how to implement the **state machine** we studied in class. It allowed us to reuse many of the **programming** concepts and methods acquired during previous courses and **SAÉ projects**. We also became familiar with the use of pointers, which was a quite new concept for us.

Another interesting aspect of the project was the discovery of new **hardware components**, especially the **ultrasonic sensor**. We did not immediately understand how to use the component but after studying the datasheet for few days we managed to make it work.

We also developed a more **rigorous approach to problem solving**. For example, we used version control techniques by **regularly saving** different versions of the program before making major changes. To make sure that we always had a functional version of the code. In addition, we **tested each function separately** before integrating it into the main program. This method helped us identify bugs more easily and improved the **overall reliability** of the system.

However, the project was not without difficulties. One of the main challenges was understanding how the matrix keypad works and how to correctly **scan** and **interpret key presses**.

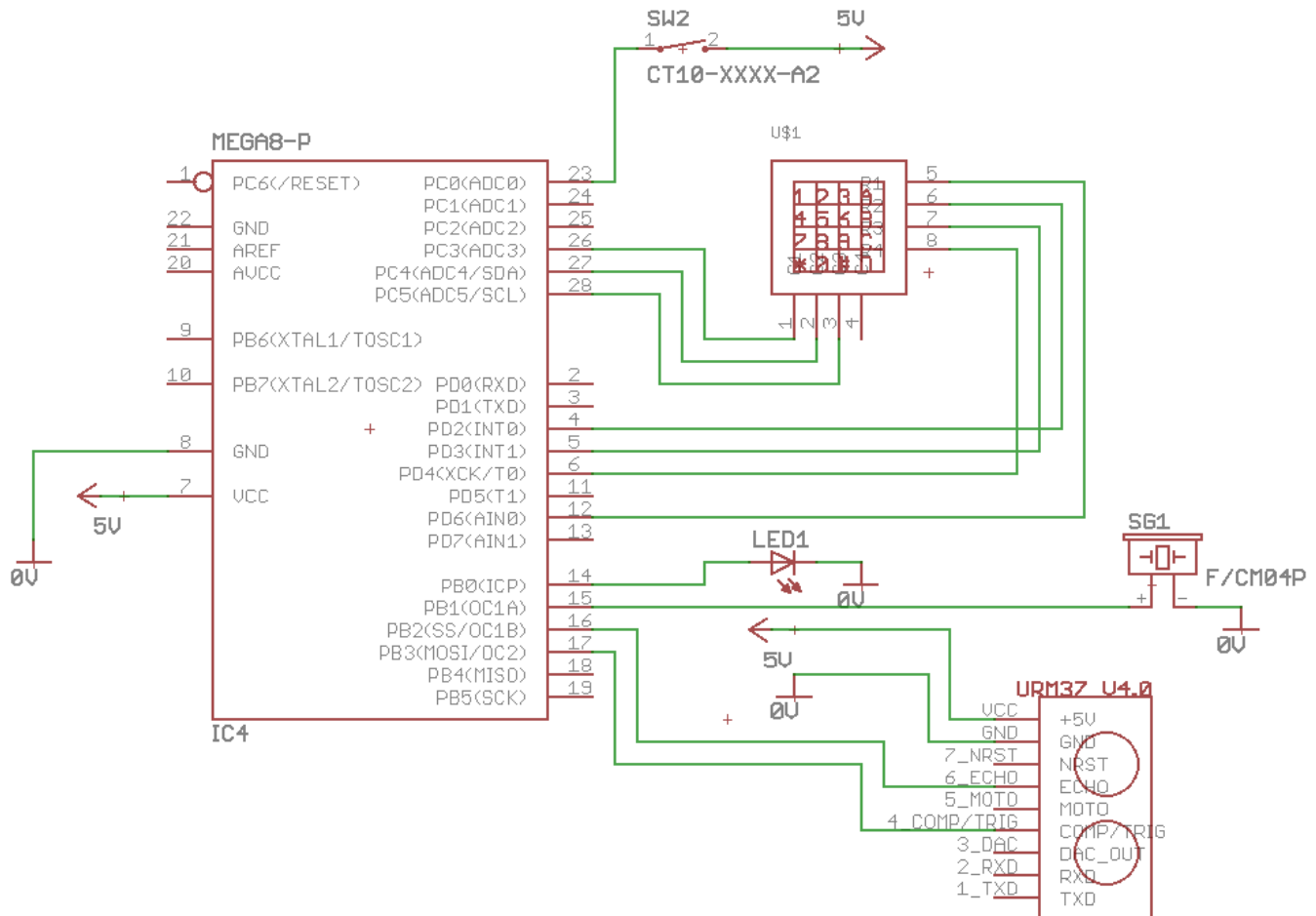
We also encountered difficulties when **reading and understanding** the **technical documentation** of the URM37 ultrasonic sensor. These documents contained many technical details and required a significant amount of time to **fully understand**.

Overall, this project allowed us to strengthen our programming skills, improve our **problem-solving methodology**, and gain practical experience with embedded systems and electronic components. Despite the challenges encountered, it was a valuable learning experience that helped us **develop** both our **technical** and **organizational abilities**.

Finally, it gave us a good look into what an **electronics and embedded/on_board system** project can look like. Very interesting as we both chose this specialisation for next year.

VII. Annexes :

Schéma structurel EAGLE :



Documentation technique ATmega328 : TIMER 1

Utilisation en mode PWM

Les bits ICNC1 et ICES1 seront positionnés à 0

ICR1 = TOP = Valeur **maximum** de comptage du Timer

OCR1A ⇒ Fixe de rapport cyclique

OCR1A = 0.5 * ICR1; // 50 % (rapport cyclique)

Output Compare Timer 1 (OC1A) ⇒ **PB1**

$$f_{OCnxPFCPWM} = \frac{f_{clk_I/O}}{2 \cdot N \cdot TOP}$$

The N variable represents the prescaler divider (1, 8, 64, 256, or 1024).

TCCR1A – Timer/Counter1 Control Register A

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR1B – Timer/Counter1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 15-3. Compare Output Mode, Phase Correct and Phase and Frequency Correct PWM⁽¹⁾

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 9 or 11: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on Compare Match when up-counting. Set OC1A/OC1B on Compare Match when downcounting.
1	1	Set OC1A/OC1B on Compare Match when up-counting. Clear OC1A/OC1B on Compare Match when downcounting.

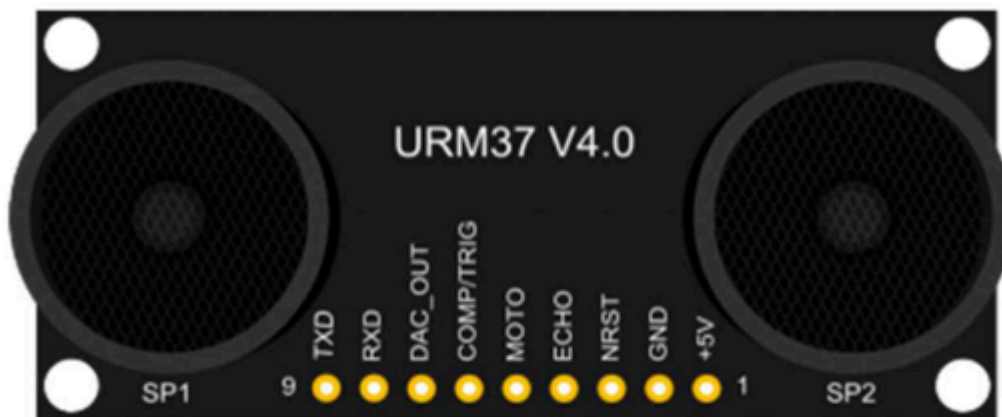
Table 15-4. Waveform Generation Mode Bit Description⁽¹⁾

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

Table 15-5. Clock Select Bit Description

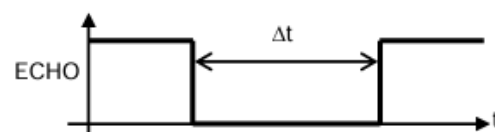
CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk _{IO} /1 (No prescaling)
0	1	0	clk _{IO} /8 (From prescaler)
0	1	1	clk _{IO} /64 (From prescaler)
1	0	0	clk _{IO} /256 (From prescaler)
1	0	1	clk _{IO} /1024 (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Documentation technique UMR 37 : UltraSonic Sensor



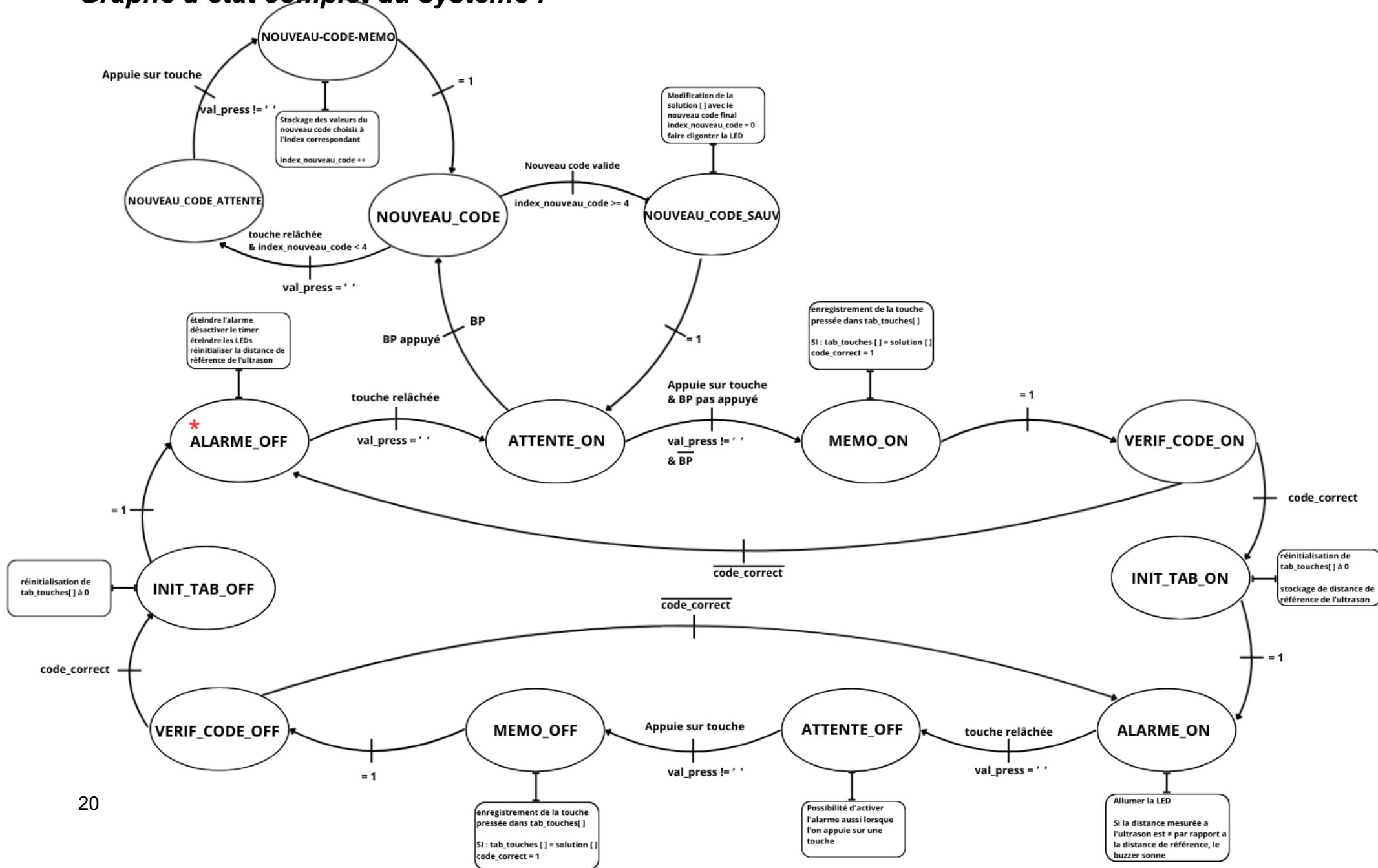
NumLabel	Description
1 VCC	Power input (reference +5V)
2 GND	Ground
3 NRST	Reset
4 ECHO	Measured distance presented by the Data Output 0-25000US by PWM pulse width, 1 CM / 50US representative
5 MOTO	Control the servo rotation angle
6 COMP/TRIG	COMP : On/OFF mode, when the detecting distance is smaller than a pre-set value, this pin pulls low./TRIG : PWM mode trigger input
7 DAC	Analog voltage output (each 6.8mV represents 1 cm)
8 RXD	RS232/TTL mode
9 TXD	RS232/TTL mode

La sortie ECHO du module permet de mesurer la distance entre le module et un obstacle : En effet, cette sortie indique cette distance en donnant un signal numérique dont la durée de l'état bas (Δt) est proportionnelle à cette distance (50 μ s/cm). Vous devez donc mesurer la durée Δt afin de connaître la distance de l'obstacle (utilisez un graphe d'état).



L'entrée COMP/TRIG permet de placer le module en veille pour éviter qu'il ne consomme. On placera aussi une LED sur la sortie ECHO afin qu'elle s'allume quand ECHO est au niveau bas.

Graphe d'état complet du système :



Graphe d'état fonction ultrason :

